



RetainHub

Sistema Inteligente de Alerta Temprana para la
Fidelización de Clientes en E-commerce

Trabajo de:	Curso de Especialización de Inteligencia Artificial y Big Data
Asignatura:	Proyecto de Inteligencia Artificial y Big Data
Autores:	Ángela Baena, Mariano Correa y Carlos Suárez
Profesor:	José Miguel Ruiz Guevara
Centro:	iFP Innovación en Formación Profesional
Año:	2026

Índice

1. Desarrollo del proyecto.....	1
2. Estructura del código.....	2
2.1. Fase 1: Preprocesamiento y Limpieza.....	2
2.2. Fase 2: Ingeniería de Características y Segmentación.....	2
2.3. Fase 3: Modelado Predictivo y Detección de Abandono.....	3
3. Metodología.....	4
3.1. Tecnologías utilizadas.....	4
3.2. Dataset (Base de datos).....	4
3.3. Repositorio de Github.....	5
3.4. Código.....	5
Fase 1: Preprocesamiento y Limpieza.....	5
1. Selección y carga de datos.....	5
2. Limpieza inicial.....	5
3. Transformación de datos.....	9
Fase 2: Ingeniería de Características y Segmentación.....	11
1. Creación de variables.....	11
2. Segmentación RFM.....	12
3. Preparación para el modelado.....	13
4. Visualización de los segmentos finales.....	14
5. Exploración del dataset preparado.....	15
Fase 3: Modelado Predictivo y Detección de Abandono.....	15
1. Carga de datos transformados.....	15
2. Corrección del desequilibrio.....	16
3. Entrenamiento del modelo.....	16
4. Generación del score de riesgo.....	16

1. Desarrollo del proyecto

RetainHub se concibe como una solución tecnológica para el sector e-commerce, diseñada para combatir la pérdida silenciosa de clientes mediante un sistema inteligente de alerta temprana. El núcleo del proyecto es la creación de un modelo predictivo capaz de identificar patrones de abandono (churn) para permitir acciones de marketing proactivas y así no perder clientes.

El desarrollo del proyecto consta de 4 fases, las cuales se explican a continuación.

1. **Fase de Ingesta y Preparación de Datos (ETL).** El primer paso crítico es el tratamiento de los datos históricos de ventas y perfiles de usuario. En esta primera fase se ha seleccionado un dataset adecuado de la plataforma de Kaggle, llamado "Ecommerce Consumer Behavior Analysis Data". A continuación, se ha realizado la limpieza y normalización de los datos. Se realiza un proceso exhaustivo de manejo de valores nulos, eliminación de duplicados y aseguramiento de que los datos tabulares sean procesables mediante la librería Pandas en un entorno Python. En esta fase, Se redacta un informe de calidad de datos y un análisis descriptivo inicial que servirá como base para el reporte final
2. **Ingeniería de Características y Análisis RFM.** Una vez depurada la información, se transforman los datos brutos en variables con valor predictivo. Se generan las métricas clave. Se programan variables fundamentales como "días desde la última compra" y "gasto total acumulado" (Valor Monetario). Posteriormente, se realiza el análisis de Recencia, Frecuencia y Valor Monetario. Este análisis no solo alimenta el modelo, sino que constituye el "Plan B" estratégico: si la predicción fallara, el equipo puede basar su estrategia de retención en esta segmentación descriptiva profunda. También, se realiza un preprocesamiento. Se codifican variables categóricas y se realiza la normalización necesaria para que los algoritmos procesen la información correctamente.
3. **Modelado Predictivo y Ciencia de Datos.** En esta fase se realiza el entrenamiento del algoritmo. Se implementan modelos de clasificación binaria ("¿Se va o se queda?"), utilizando algoritmos como Random Forest o Regresión Logística. La tarea más compleja de esta fase es la gestión del desequilibrio de datos, dado que habitualmente existen muchos más clientes que permanecen que aquellos que abandonan. Una vez realizado el entrenamiento del modelo, se calcula la precisión y se asigna a cada cliente un "score" de riesgo de 0 a 100%.
4. **Análisis de Negocio y Entrega de Resultados.** La fase final, traduce los datos técnicos en valor accionable para la empresa. Se diseñan gráficos que ilustran los patrones de abandono y se genera el reporte "Top 100", listando a los clientes con mayor probabilidad de fuga. Se redactan recomendaciones específicas para los equipos de marketing y gestión de experiencia del cliente, orientadas a la creación de campañas dirigidas y ofertas personalizadas. Por último, se asegura que el entorno esté debidamente organizado y documentado para la entrega final del proyecto.

2. Estructura del código

El proyecto se desarrolla siguiendo el ciclo de vida de un proyecto de análisis de datos, estructurado en los siguientes bloques.

2.1. Fase 1: Preprocesamiento y Limpieza

En esta etapa, se transforman los datos brutos de Kaggle en un conjunto de datos consistente y libre de errores.

- **Carga y normalización:** descarga automatizada del dataset y estandarización de los nombres de las columnas a minúsculas (snake_case) para facilitar la programación.
- **Limpieza de datos críticos:** conversión de tipos de datos, eliminación de registros nulos y gestión de duplicados.
- **Tratamiento de outliers:** identificación de valores atípicos mediante el método del Rango Intercuartílico (IQR).
- **Estandarización de texto:** limpieza de espacios en blanco y normalización de etiquetas categóricas (como el género o la categoría de compra) para evitar inconsistencias.

2.2. Fase 2: Ingeniería de Características y Segmentación

Transformación de los datos limpios en métricas de negocio para entender el comportamiento del consumidor.

- **Cálculo de métricas RFM:** creación de las variables de Recencia (días desde la última compra), Frecuencia (número de pedidos) y Valor Monetario (gasto total).
- **Segmentación de clientes:** clasificación de usuarios en grupos (Campeones, Leales, En Riesgo) utilizando percentiles sobre el Score RFM.
- **Preparación para modelado:** conversión de variables categóricas en vectores numéricos.
 - **Escalado:** normalización de variables numéricas (age, purchase_amount) mediante StandardScaler para asegurar que todas tengan el mismo peso en el modelo.

2.3. Fase 3: Modelado Predictivo y Detección de Abandono

Implementación de Inteligencia Artificial para predecir el comportamiento futuro de los clientes.

- **Definición del target (Churn):** creación de una variable binaria donde 1 representa a un cliente con alta probabilidad de abandonar la marca.
- **Balanceo de datos (SMOTE):** aplicación de la técnica de sobremuestreo para equilibrar las clases, evitando que el modelo ignore a los clientes que se fugan.
- **Entrenamiento (Random Forest):** uso de un bosque aleatorio para capturar patrones complejos en los datos de entrenamiento.
- **Generación de resultados:** cálculo de probabilidades de fuga y exportación de un archivo CSV llamado `top_100_riesgo.csv` con los 100 clientes con mayor probabilidad de abandono, para su uso en acciones de marketing preventivo.

3. Metodología

3.1. Tecnologías utilizadas

Para la realización de este proyecto, se ha necesitado las siguientes herramientas:

- **Python 3.13**
- **Pandas & NumPy**: Manipulación y análisis de datos.
- **Seaborn & Matplotlib**: Visualización estadística.
- **Scikit-Learn**: Algoritmos de Machine Learning y preprocesamiento.
- **Imbalanced-Learn (SMOTE)**: Gestión de datasets desequilibrados.

3.2. Dataset (Base de datos)

El dataset utilizado para este proyecto se ha obtenido de la plataforma Kaggle. El escogido se llama “Ecommerce Consumer Behavior Analysis Data”. Fue escogido ya que para responder a la necesidad de construir un sistema de alerta temprana robusto y preciso es el más adecuado por una serie de motivos que se exponen a continuación.

- **Riqueza de los datos.** Para predecir el abandono, no basta con saber qué se compró. Este dataset ofrece una visión completa del consumidor al integrar tres tipos de datos distintos. Entre ellos están los datos transaccionales como el monto gastado y la categoría del producto, permitiendo identificar el valor monetario del cliente, datos demográficos como la edad, género, nivel de ingresos y educación y datos sobre el comportamiento y la satisfacción del cliente.
- **Viabilidad para Modelos de Machine Learning.** El dataset presenta unas características técnicas que facilitan el entrenamiento del modelo ya que posee una cantidad de registros suficiente para garantizar la viabilidad estadística sin perder la capacidad de generalización. Existen variables de periodicidad que permiten realizar análisis temporales para detectar cambios en la frecuencia de compra, que es el indicador principal de un cliente que está a punto de abandonar. Además, la organización de los datos permite una limpieza y transformación eficiente, reduciendo el ruido y enfocándose en la ingeniería de características.
- **Alineación con el objetivo del proyecto.** El propósito de RetainHub es la retención. Este dataset permite responder preguntas críticas para el negocio como el perfil de cliente que gasta más pero compra con menos frecuencia o si existe una correlación entre el nivel de ingresos y el riesgo de abandono.

En resumen, se ha seleccionado este dataset porque proporciona la trazabilidad completa del viaje del cliente, desde quién es hasta cómo compra, lo cual es el combustible necesario para que un sistema de alerta temprana sea realmente preventivo.

3.3. Repositorio de Github

El código se encontrará en el siguiente repositorio de GitHub: [repositorio](#)

3.4. Código

A continuación se presenta el código del proyecto separado por las distintas fases definidas en el apartado 2, y con su respectiva imagen de lo que sale por pantalla.

Fase 1: Preprocesamiento y Limpieza

1. Selección y carga de datos

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import os
import kagglehub

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, roc_auc_score
from imblearn.over_sampling import SMOTE # Herramienta para el desequilibrio

path =
kagglehub.dataset_download("salahuddinahmedshuvo/ecommerce-consumer-behavior-analysis-data")

for dirname, _, filenames in os.walk(path):
    for filename in filenames:
        print(os.path.join(dirname, filename))
        dataset = os.path.join(dirname, filename)
```

2. Limpieza inicial

```
# Guardo los datos en una variable e imprimo las columnas que tiene el dataset
datos = pd.read_csv(dataset)
print(datos.columns)

# Veo los valores nulos que hay y el porcentaje de ellos por columna
print("Valores nulos antes de borrar:")
print(datos.isnull().sum())
print (datos.isnull().mean() * 100)

# Comprobación de registros duplicados
duplicados = datos.duplicated().sum()
print(f"Registros duplicados encontrados: {duplicados}")

if duplicados > 0:
```

```

    datos = datos.drop_duplicates()
    print("Duplicados eliminados correctamente.")

# Verificación final de la forma del dataset tras la limpieza
print(f"Dimensiones finales del dataset: {datos.shape}")

```

```

Index(['Customer_ID', 'Age', 'Gender', 'Income_Level', 'Marital_Status',
      'Education_Level', 'Occupation', 'Location', 'Purchase_Category',
      'Purchase_Amount', 'Frequency_of_Purchase', 'Purchase_Channel',
      'Brand_Loyalty', 'Product_Rating',
      'Time_Spent_on_Product_Research(hours)', 'Social_Media_Influence',
      'Discount_Sensitivity', 'Return_Rate', 'Customer_Satisfaction',
      'Engagement_with_Ads', 'Device_Used_for_Shopping', 'Payment_Method',
      'Time_of_Purchase', 'Discount_Used', 'Customer_Loyalty_Program_Member',
      'Purchase_Intent', 'Shipping_Preference', 'Time_to_Decision'],
      dtype='object')
Valores nulos antes de borrar:
Customer_ID          0
Age                  0
Gender               0
Income_Level         0
Marital_Status       0
Education_Level      0
Occupation           0
Location             0
Purchase_Category    0
Purchase_Amount      0
Frequency_of_Purchase 0
Purchase_Channel     0
Brand_Loyalty        0
Product_Rating       0
...
Time_to_Decision     0.0
dtype: float64
Registros duplicados encontrados: 0
Dimensiones finales del dataset: (1000, 28)

```

```

# Normalizamos nombres de columnas a minúsculas
datos.columns = [col.lower().replace(' ', '_').strip() for col in datos.columns]

# Limpieza agresiva de la columna purchase_amount
# Eliminamos símbolos de moneda, comas y espacios si existieran
if datos['purchase_amount'].dtype == 'object':
    datos['purchase_amount'] = datos['purchase_amount'].str.replace('$', '',
regex=False).str.replace(',', '', regex=False).str.strip()

# Convertimos a numérico (los que no se puedan convertir serán NaN)
datos['purchase_amount'] = pd.to_numeric(datos['purchase_amount'], errors='coerce')

# Verificamos cuántos datos sobrevivieron antes de borrar nulos
nulos_generados = datos['purchase_amount'].isnull().sum()

```



```

print(f"Valores que no se pudieron convertir a número: {nulos_generados}")

# Borramos nulos
datos = datos.dropna(subset=['purchase_amount'])

# Cálculo de IQR
if not datos.empty:
    Q1 = datos['purchase_amount'].quantile(0.25)
    Q3 = datos['purchase_amount'].quantile(0.75)
    IQR = Q3 - Q1

    limite_inferior = Q1 - 1.5 * IQR
    limite_superior = Q3 + 1.5 * IQR

    print(f"Tipo de dato: {datos['purchase_amount'].dtype}")
    print(f"Filas útiles: {len(datos)}")
    print(f"Límites estadísticos: {limite_inferior:.2f} a {limite_superior:.2f}")

    # Identificar outliers
    outliers = datos[(datos['purchase_amount'] < limite_inferior) |
                     (datos['purchase_amount'] > limite_superior)]
    print(f"Número de transacciones atípicas detectadas: {len(outliers)}")
else:
    print(";ERROR CRÍTICO! El dataset se quedó vacío tras procesar
Purchase_Amount.")

```

```

Valores que no se pudieron convertir a número: 0
Tipo de dato: float64
Filas útiles: 1000
Límites estadísticos: -177.89 a 729.10
Número de transacciones atípicas detectadas: 0

```

```

# Validar que no existan montos de compra negativos (nombres en minúsculas)
conteo_negativos = (datos['purchase_amount'] <= 0).sum()
datos = datos[datos['purchase_amount'] > 0]

# Validar rango de edad (clientes entre 18 y 100 años)
conteo_edad_erronea = ((datos['age'] < 18) | (datos['age'] > 100)).sum()
datos = datos[(datos['age'] >= 18) & (datos['age'] <= 100)]

print(f"Registros eliminados por montos <= 0: {conteo_negativos}")
print(f"Registros eliminados por edades fuera de rango: {conteo_edad_erronea}")
print(f"Dimensiones actuales del dataset: {datos.shape}")

```

```
Registros eliminados por montos <= 0: 0
Registros eliminados por edades fuera de rango: 0
Dimensiones actuales del dataset: (1000, 28)
```

```
# Eliminar espacios innecesarios en columnas de texto
columnas_obj = datos.select_dtypes(include=['object']).columns
for col in columnas_obj:
    datos[col] = datos[col].str.strip()

# Identificar y normalizar placeholders de 'nulos ocultos'
placeholders = ['?', 'n/a', 'Unknown', 'unknown', 'None']
datos.replace(placeholders, np.nan, inplace=True)

# Comprobar si han aparecido nuevos nulos reales tras la limpieza
print("Nuevos valores nulos detectados tras estandarización:")
print(datos.isnull().sum())

print("Nuevos valores nulos detectados tras estandarización:")
nulos = datos.isnull().sum()
print(nulos[nulos > 0]) # Solo mostramos las columnas que tienen nulos para hacer
resumen
```

Nuevos valores nulos detectados tras estandarización:

customer_id	0
age	0
gender	0
income_level	0
marital_status	0
education_level	0
occupation	0
location	0
purchase_category	0
purchase_amount	0
frequency_of_purchase	0
purchase_channel	0
brand_loyalty	0
product_rating	0
time_spent_on_product_research(hours)	0
social_media_influence	247
discount_sensitivity	0
return_rate	0
customer_satisfaction	0
engagement_with_ads	256
device_used_for_shopping	0
payment_method	0
time_of_purchase	0
discount_used	0
...	

Nuevos valores nulos detectados tras estandarización:

social_media_influence	247
engagement_with_ads	256

dtype: int64

3. Transformación de datos

```
# Convertir columnas categóricas para optimizar procesamiento
categoricas = ['gender', 'income_level', 'marital_status', 'education_level',
               'occupation', 'location', 'purchase_category', 'purchase_channel',
               'payment_method', 'discount_used',
               'customer_loyalty_program_member', 'shipping_preference']

for col in categoricas:
    datos[col] = datos[col].astype('category')

# Aunque el dataset de Kaggle viene con Time_of_Purchase como string,
# se convierte si el formato lo permite para futuros análisis de periodicidad.
datos['time_of_purchase'] = pd.to_datetime(datos['time_of_purchase'])

print("Tipos de datos actualizados:")
print(datos.dtypes)
```

```

Tipos de datos actualizados:
customer_id          object
age                  int64
gender               category
income_level         category
marital_status       category
education_level      category
occupation           category
location             category
purchase_category    category
purchase_amount      float64
frequency_of_purchase int64
purchase_channel     category
brand_loyalty        int64
product_rating       int64
time_spent_on_product_research(hours) float64
social_media_influence object
discount_sensitivity object
return_rate          int64
customer_satisfaction int64
engagement_with_ads  object
device_used_for_shopping object
payment_method       category
time_of_purchase     datetime64[ns]
discount_used        category
...
purchase_intent      object
shipping_preference  category
time_to_decision     int64
dtype: object

```

```

# Convertir todos los nombres de columnas a minúsculas y sustituir espacios por
# guiones bajos
datos.columns = [col.lower().replace(' ', '_') for col in datos.columns]

print("Nuevos nombres de columnas:")
print(datos.columns)

```

```

Nuevos nombres de columnas:
Index(['customer_id', 'age', 'gender', 'income_level', 'marital_status',
      'education_level', 'occupation', 'location', 'purchase_category',
      'purchase_amount', 'frequency_of_purchase', 'purchase_channel',
      'brand_loyalty', 'product_rating',
      'time_spent_on_product_research(hours)', 'social_media_influence',
      'discount_sensitivity', 'return_rate', 'customer_satisfaction',
      'engagement_with_ads', 'device_used_for_shopping', 'payment_method',
      'time_of_purchase', 'discount_used', 'customer_loyalty_program_member',
      'purchase_intent', 'shipping_preference', 'time_to_decision'],
      dtype='object')

```

```
# Unificar formatos de texto en columnas categóricas
columnas_a_unificar = ['gender', 'location', 'purchase_category']

for col in columnas_a_unificar:
    if col in datos.columns:
        # Convertir a título (Primera letra mayúscula, resto minúscula)
        datos[col] = datos[col].astype(str).str.title()

print("Etiquetas de texto unificadas.")
```

Etiquetas de texto unificadas.

Fase 2: Ingeniería de Características y Segmentación

1. Creación de variables

```
# Definimos la fecha de referencia (un día después de la última compra registrada)
fecha_referencia = datos['time_of_purchase'].max() + pd.Timedelta(days=1)

# Agrupamos por cliente para crear las métricas base
rfm = datos.groupby('customer_id').agg({
    'time_of_purchase': lambda x: (fecha_referencia - x.max()).days, # Recencia
    'purchase_amount': ['count', 'sum']                               # Frecuencia y
                                                                    Valor Monetario
})

# Renombrar columnas para claridad
rfm.columns = ['recencia', 'frecuencia', 'monetario']
rfm = rfm.reset_index()

print("Métricas RFM creadas con éxito.")
print(rfm.head())
```

Métricas RFM creadas con éxito.

	customer_id	recencia	frecuencia	monetario
0	00-107-4749	103	1	178.04
1	00-149-4481	331	1	464.76
2	00-264-3797	258	1	128.01
3	00-265-0556	60	1	68.02
4	00-275-9990	186	1	302.74

```
# Aseguramos que no haya nulos en el ID de cliente ni en la fecha
datos_limpios = datos.dropna(subset=['customer_id', 'time_of_purchase']).copy()
```

```
# Forzamos la conversión a datetime y eliminamos fallos
datos_limpios['time_of_purchase'] =
pd.to_datetime(datos_limpios['time_of_purchase'], errors='coerce')
datos_limpios = datos_limpios.dropna(subset=['time_of_purchase'])

# Fecha de referencia
fecha_ref = datos_limpios['time_of_purchase'].max() + pd.Timedelta(days=1)

# Agrupación
rfm = datos_limpios.groupby('customer_id').agg({
    'time_of_purchase': lambda x: (fecha_ref - x.max()).days,
    'purchase_amount': ['count', 'sum']
})

# Renombrar columnas de forma simple
rfm.columns = ['recencia', 'frecuencia', 'monetario']
rfm = rfm.reset_index()

print(f"Valores nulos en RFM: {rfm.isnull().sum().sum()}")
print(rfm.head())
```

```
Valores nulos en RFM: 0
```

	customer_id	recencia	frecuencia	monetario
0	00-107-4749	103	1	178.04
1	00-149-4481	331	1	464.76
2	00-264-3797	258	1	128.01
3	00-265-0556	60	1	68.02
4	00-275-9990	186	1	302.74

2. Segmentación RFM

```
# Calcular el ranking porcentual (de 0 a 1) para cada métrica
# rank(pct=True) calcula en qué percentil está cada cliente
r_pct = rfm['recencia'].rank(pct=True, method='first')
f_pct = rfm['frecuencia'].rank(pct=True, method='first')
m_pct = rfm['monetario'].rank(pct=True, method='first')

# Convertir percentiles a notas del 1 al 5
# Recencia: Menor valor es mejor, por eso restamos del 6
rfm['R'] = (r_pct.apply(lambda x: 1 if x <= 0.2 else 2 if x <= 0.4 else 3 if x <=
0.6 else 4 if x <= 0.8 else 5))
rfm['R'] = 6 - rfm['R'] # Invertimos: 1 es lejos, 5 es reciente

rfm['F'] = (f_pct.apply(lambda x: 1 if x <= 0.2 else 2 if x <= 0.4 else 3 if x <=
0.6 else 4 if x <= 0.8 else 5))
rfm['M'] = (m_pct.apply(lambda x: 1 if x <= 0.2 else 2 if x <= 0.4 else 3 if x <=
0.6 else 4 if x <= 0.8 else 5))
```

```
# Score combinado
rfm['rfm_score'] = rfm['R'].astype(str) + rfm['F'].astype(str) +
rfm['M'].astype(str)

# Etiquetado lógico
def segmentar(s):
    if s[0] >= '4' and s[1] >= '4': return 'Campeones'
    if s[0] >= '3' and s[2] >= '3': return 'Leales / Potenciales'
    if s[0] <= '2': return 'En Riesgo / Perdidos'
    return 'Otros'

rfm['segmento_cliente'] = rfm['rfm_score'].apply(segmentar)

print("Segmentación terminada con éxito.")
print(rfm['segmento_cliente'].value_counts())
```

```
Segmentación terminada con éxito.
segmento_cliente
En Riesgo / Perdidos    400
Leales / Potenciales    260
Otros                   183
Campeones               157
Name: count, dtype: int64
```

3. Preparación para el modelado

```
# LIMPIEZA DE LLAVES Y UNIÓN
# Aseguramos que los IDs no tengan espacios y sean del mismo tipo (string)
datos['customer_id'] = datos['customer_id'].astype(str).str.strip()
rfm['customer_id'] = rfm['customer_id'].astype(str).str.strip()

# Realizamos la unión
datos_final = datos.merge(rfm[['customer_id', 'segmento_cliente', 'rfm_score']],
on='customer_id', how='left')

# Verificación de seguridad
if datos_final.shape[0] == 0:
    print(";ERROR: El merge resultó en 0 filas! Revisa si 'customer_id' existe en
ambos dataframes.")
else:
    # CODIFICACIÓN (One-Hot Encoding)
    # Seleccionamos solo las columnas que realmente existen en el dataframe
    columnas_disponibles = datos_final.columns.tolist()
    columnas_cat = [c for c in ['gender', 'income_level', 'purchase_category',
                                'purchase_channel', 'payment_method',
                                'segmento_cliente']
                    if c in columnas_disponibles]
```

```

    datos_preparados = pd.get_dummies(datos_final, columns=columnas_cat,
drop_first=True)

    # TRATAMIENTO DE NULOS ANTES DE ESCALAR
    # El Scaler falla si hay nulos. Llenamos nulos en edad con la mediana y en
monto con 0.
    datos_preparados['age'] =
datos_preparados['age'].fillna(datos_preparados['age'].median())
    datos_preparados['purchase_amount'] =
datos_preparados['purchase_amount'].fillna(0)

    # NORMALIZACIÓN (Escalado)
    from sklearn.preprocessing import StandardScaler

    scaler = StandardScaler()
    cols_a_escalor = ['age', 'purchase_amount']

    # Solo escalamos si el dataframe tiene datos
    if not datos_preparados.empty:
        datos_preparados[cols_a_escalor] =
scaler.fit_transform(datos_preparados[cols_a_escalor])

    # LIMPIEZA FINAL DE COLUMNAS
    columnas_sobrantes = ['customer_id', 'time_of_purchase', 'location',
'rfm_score']
    datos_preparados = datos_preparados.drop(columns=[c for c in
columnas_sobrantes if c in datos_preparados.columns])

    print("----- PROCESO FINALIZADO -----")
    print(f"Dataset listo. Filas: {datos_preparados.shape[0]}, Columnas:
{datos_preparados.shape[1]}")
    else:
        print("Error: El dataset preparado quedó vacío.")

```

```

----- PROCESO FINALIZADO -----
Dataset listo. Filas: 1000, Columnas: 60

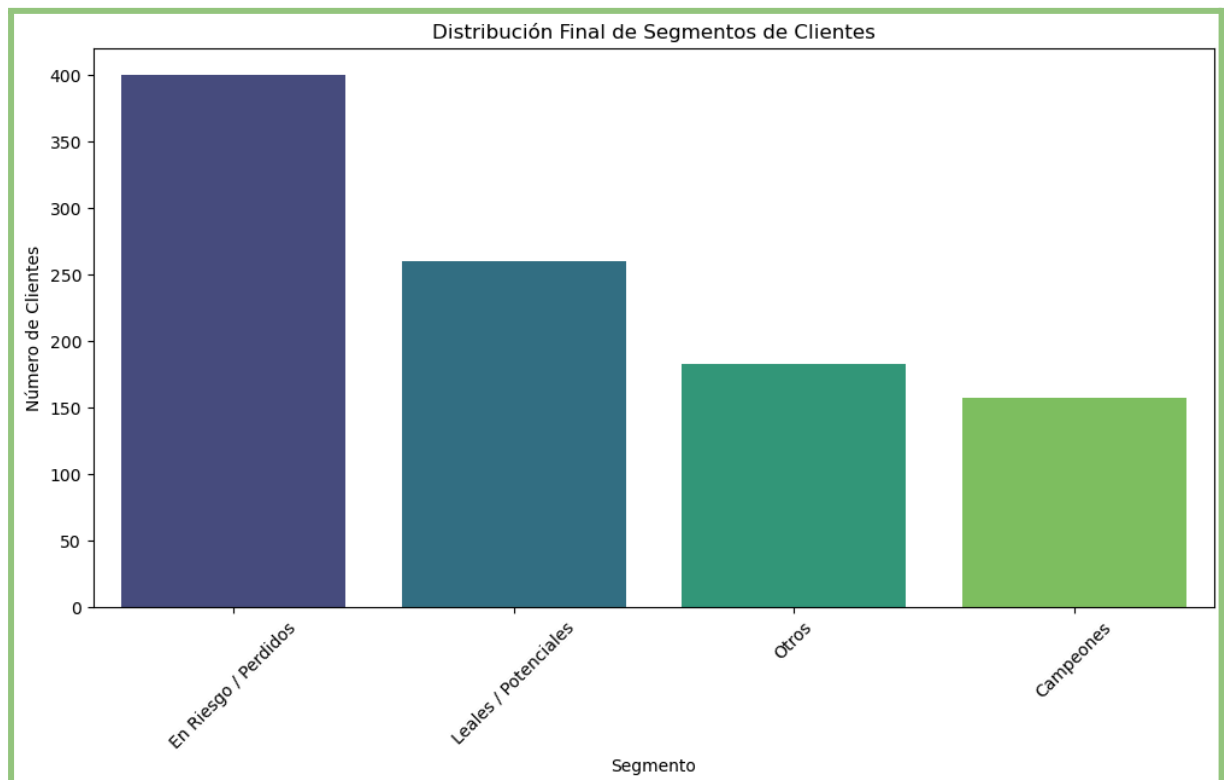
```

4. Visualización de los segmentos finales

```

plt.figure(figsize=(12, 6))
sns.countplot(data=rfm, x='segmento_cliente', palette='viridis',
order=rfm['segmento_cliente'].value_counts().index)
plt.title('Distribución Final de Segmentos de Clientes')
plt.xlabel('Segmento')
plt.ylabel('Número de Clientes')
plt.xticks(rotation=45)
plt.show()

```

5. Exploración del dataset preparado

```
# Guardamos el dataset listo para Machine Learning
datos_preparados.to_csv('processed_data.csv', index=False)
print("Archivo 'processed_data.csv' guardado con éxito.")
```

```
Archivo 'processed_data.csv' guardado con éxito.
```

Fase 3: Modelado Predictivo y Detección de Abandono

1. Carga de datos transformados

```
# Carga y Limpieza de columnas no numéricas
df = pd.read_csv('processed_data.csv')

# Creamos el target 'y'
target_df = rfm[['customer_id', 'segmento_cliente']].copy()
target_df['churn'] = target_df['segmento_cliente'].apply(lambda x: 1 if x == 'En
Riesgo / Perdidos' else 0)

# Solo nos quedamos con las columnas que son números (float, int) o booleanos
X = df.select_dtypes(include=[np.number, 'bool']).copy()
```

```
print(f"Columnas numéricas detectadas para el modelo: {X.columns.tolist()}")

y = target_df['churn']

# Alineación por si acaso los tamaños difieren
X = X.iloc[:len(y)]
y = y.iloc[:len(X)]

# División
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42, stratify=y)
```

```
Columnas numéricas detectadas para el modelo: ['age', 'purchase_amount', 'frequency_of_purchase', 'brand_1
```

2. Corrección del desequilibrio

```
# Aplicar SMOTE
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

print(f"Nueva distribución equilibrada: \n{y_train_res.value_counts()}")
```

```
Nueva distribución equilibrada:
churn
0    480
1    480
Name: count, dtype: int64
```

3. Entrenamiento del modelo

```
# Creamos el modelo
model = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)

# Entrenamos con los datos equilibrados por SMOTE
model.fit(X_train_res, y_train_res)

print("Modelo entrenado exitosamente con datos balanceados.")
```

```
Modelo entrenado exitosamente con datos balanceados.
```

4. Generación del score de riesgo

```
# Calculamos la probabilidad de abandono (clase 1)
```

```

probabilidades = model.predict_proba(X_test)[: , 1] # predict_proba nos da dos
columnas: [prob_quedarse, prob_irse]. Tomamos la segunda [:, 1]

# Recuperamos los IDs de cliente originales
ids_clientes = rfm.loc[X_test.index, 'customer_id'].values # Usamos el índice de
X_test para buscar el customer_id en la tabla rfm original

# Creamos un DataFrame con los resultados detallados
resultados = pd.DataFrame({
    'id_cliente': ids_clientes,
    'probabilidad_churn_%': (probabilidades * 100).round(2)
})

# Generamos el top 100 clientes con mayor riesgo de fuga
top_100_riesgo = resultados.sort_values(by='probabilidad_churn_%',
ascending=False).head(100)

# Guardamos el archivo
top_100_riesgo.to_csv('top_100_riesgo_clientes.csv', index=False)

print("----- REPORTE DE RIESGO GENERADO -----")
print(f"Se han identificado {len(top_100_riesgo)} clientes prioritarios.")
print("\nPrimeros 10 clientes en riesgo máximo:")
print(top_100_riesgo.head(10))

```

```

----- REPORTE DE RIESGO GENERADO -----
Se han identificado 100 clientes prioritarios.

Primeros 10 clientes en riesgo máximo:
   id_cliente  probabilidad_churn_%
185  93-093-0556                63.80
180  35-418-0288                62.82
176  63-631-8261                62.51
115  79-939-5393                62.03
183  75-915-9803                61.32
5    15-711-4107                60.30
47   78-905-2660                60.27
168  10-330-6387                60.23
45   36-214-1586                59.64
86   34-788-8387                59.35

```

